

AI Vision for Blind: An IoT-Based Real-Time Scene Description System Using Large Vision-Language Models and Text-to-Speech

SAJAL PATIL¹, ADITI PORE², AND DR. NILIMA R. KOLHARE³

Department of Electronics and Telecommunication Engineering, College of Engineering Pune (COEP), Pune, India

Microcontrollers and its Applications (MCA) Laboratory

dipakraops24.extc@coeptech.ac.in | alankarpa24.extc@coeptech.ac.in | nek.extc@coeptech.ac.in

ABSTRACT Visual impairment affects over 285 million people worldwide, significantly limiting their ability to perceive and interact with their environment independently. This paper presents an embedded, AI-powered assistive system designed to provide real-time auditory scene descriptions for visually impaired individuals. The proposed system integrates an AI-Thinker ESP32-CAM microcontroller with a MAX98357A I2S digital amplifier, a 0.96-inch SSD1306 OLED display, and a single push-button interface. Upon pressing the button, the system captures a JPEG image, encodes it to Base64, and transmits it over Wi-Fi to the Groq cloud API, which employs the Meta LLaMA-4 Scout large vision-language model to generate a concise, natural-language description of the scene. The textual response is subsequently chunked and converted to synthesized speech using the Google Translate Text-to-Speech (TTS) service, played back through the I2S amplifier in real time, while simultaneously scrolling the description on the OLED display. Designed for low cost and portability, the system achieves end-to-end latency of approximately 4–8 seconds and requires no specialized computing hardware beyond a standard Wi-Fi connection. Experimental results demonstrate accurate and contextually meaningful scene descriptions across diverse environments, offering a practical and scalable solution for assistive vision technology.

INDEX TERMS Assistive technology, ESP32-CAM, Groq API, Internet of Things (IoT), large vision-language model, LLaMA-4, real-time scene description, text-to-speech, visual impairment.

I. INTRODUCTION

Visual impairment remains one of the most debilitating disabilities globally. According to the World Health Organization (WHO), approximately 285 million people worldwide suffer from some form of visual impairment, of whom 39 million are completely blind [1]. These individuals face significant challenges in performing everyday tasks such as identifying objects, reading labels, navigating unfamiliar environments, and recognizing faces—tasks that sighted individuals perform effortlessly through vision.

Traditional assistive devices such as white canes, guide dogs, and Braille readers address mobility and textual reading needs but do not provide comprehensive real-time visual scene understanding. The rapid advancement of Artificial Intelligence (AI), particularly in the domain of large vision-language models (VLMs), has opened new opportunities to bridge this gap by enabling machines to describe visual content in natural language.

Recent years have witnessed remarkable progress in multimodal AI systems capable of simultaneously processing visual and textual information. Models such as Meta LLaMA-4, Google Gemini, and OpenAI GPT-4 Vision can generate coherent, context-aware descriptions of images, rivaling human-level performance on visual understanding benchmarks [2]. However, most existing implementations of these models require powerful cloud servers or GPU-equipped edge devices, making them inaccessible for resource-constrained, wearable assistive applications.

This paper presents AI Vision for Blind, a compact, cost-effective, and fully embedded assistive system built around the ESP32-CAM microcontroller. The system integrates hardware-level image capture, cloud-based AI inference through the Groq API, and real-time text-to-speech (TTS) output via the Google Translate TTS service—all within a device that can be powered from a USB power bank. No dedicated GPU, laptop, or smartphone is required to operate the system.

The remainder of this paper is organized as follows: Section II states the project objectives; Section III presents the system architecture and block diagram; Section IV details the hardware components; Section V describes the circuit connections; Section VI covers the software design; Section VII explains the working principle; Section VIII provides a mathematical analysis; Section IX discusses results; Section X concludes the paper.

II. OBJECTIVES OF THE PROJECT

The primary objectives of this project are:

1. **Real-Time Scene Description:** Develop an embedded system capable of capturing an image and generating a natural-language description within 4–8 seconds, suitable for real-time use.
2. **Accessibility-First Design:** Create a single-button interface requiring zero technical knowledge, making the system immediately usable by visually impaired individuals without training.
3. **Cost Optimization:** Use exclusively low-cost commercial hardware (total Bill of Materials < ₹1,500 / ~USD 18) to ensure affordability in developing economies.
4. **Audio-Visual Feedback:** Provide simultaneous OLED text scrolling and TTS speech output so that caregivers or sighted companions can also read the description.
5. **Robust Connectivity:** Implement a Wi-Fi reconnection and watchdog mechanism ensuring uninterrupted operation across multiple capture cycles.

- Cancel Mechanism:** Support a long-press cancel action allowing users to interrupt and restart the pipeline at any stage without a reboot.

III. SYSTEM ARCHITECTURE

A. Block Diagram

The system consists of six functional blocks interconnected through wired and wireless communication interfaces. The high-level block diagram is presented in Fig. 1.

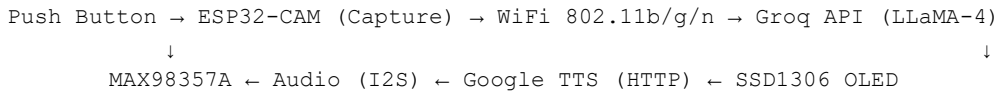


Fig. 1. Block diagram of the AI Vision for Blind system.

The pipeline flows in two stages. In the upstream stage, the user presses the push button, triggering the ESP32-CAM to capture a JPEG image. The image is Base64-encoded in firmware and transmitted as an HTTP POST request to the Groq API via the on-chip Wi-Fi module. In the downstream stage, the API response (natural-language description) is parsed, chunked into ≤ 200 -character segments, sent to Google Translate TTS, and played back in sequence through the MAX98357A I2S amplifier while the OLED display scrolls the description.

B. Data Flow

The data flow follows: Button $\rightarrow$ ESP32-CAM (JPEG Capture) $\rightarrow$ Base64 Encode $\rightarrow$ Wi-Fi $\rightarrow$ Groq API (LLaMA-4) $\rightarrow$ Text Response $\rightarrow$ TTS Chunking $\rightarrow$ Google TTS $\rightarrow$ I2S $\rightarrow$ Amplifier $\rightarrow$ Speaker. Simultaneously, the text is rendered on the SSD1306 OLED.

IV. HARDWARE DESCRIPTION

A. ESP32-CAM (AI-Thinker Module)

The AI-Thinker ESP32-CAM integrates an ESP32-S SoC with a 2 MP OV2640 camera sensor, 4 MB PSRAM, 4 MB flash, and a PCB-trace Wi-Fi antenna on a 27×40 mm board. Operating at 240 MHz dual-core Xtensa LX6, it provides sufficient processing power to perform Base64 encoding (≈ 40 – 60 ms for a VGA JPEG) and manage HTTP/HTTPS sessions concurrently. For this project, the camera is configured at VGA resolution (640×480) with JPEG quality factor 10, yielding images of 25–45 KB—a balance between visual detail and API latency.

B. MAX98357A I2S Amplifier

The MAX98357A is a filterless Class D audio amplifier with integrated I2S digital audio decoder, capable of delivering up to 3.2 W into 4Ω at 5 V. It accepts a standard I2S data stream (BCLK, LRC, DIN) directly from the ESP32, eliminating the need for a separate DAC. With the SD pin tied to 3.3 V, the amplifier operates in always-on mode at a fixed 9 dB gain. Its filterless output is suitable for small $4 \Omega / 8 \Omega$ speakers commonly used in assistive devices.

C. SSD1306 OLED Display (0.96")

The 128×64 pixel monochrome OLED display communicates over I2C (SCL: GPIO 14, SDA: GPIO 15) at 400 kHz. It serves dual roles: displaying status messages during processing and scrolling the AI-generated description at a rate of one line per 1.5 s to match speech pacing. At text size 1 (6×8 px per character), each line holds 21 characters; the display shows 7 lines simultaneously within the bordered viewport.

D. Push Button

A tactile push button connected to GPIO 3 (configured as INPUT_PULLUP) triggers the capture pipeline on a short press. GPIO 3 is normally used as UART RX on ESP32, but by disabling the hardware serial interface in the firmware, this pin is reclaimed as a general-purpose input. A long press (> 800 ms) activates the cancel mechanism, stopping audio playback and returning the system to the ready state.

V. CIRCUIT DESCRIPTION

All components are connected directly to the ESP32-CAM GPIO headers. No external power regulation is required beyond the 3.3 V and 5 V rails available on the module. Table I summarizes the complete pin mapping.

TABLE I
Pin Connections for All Hardware Components

Component	ESP32-CAM GPIO	Notes
OLED SCL	GPIO 14	I2C Clock (Wire)
OLED SDA	GPIO 15	I2C Data (Wire)
Capture Button	GPIO 3	Active LOW, INPUT_PULLUP
Flash LED	GPIO 4	Auto ON during capture
I2S BCLK	GPIO 12	MAX98357A Bit Clock
I2S LRC	GPIO 2	MAX98357A Word Select
I2S DOUT	GPIO 13	MAX98357A Data In
MAX98357A VIN	5V Rail	Power Supply

Component	ESP32-CAM GPIO	Notes
MAX98357A SD	3.3V Rail	Always ON mode

The I2C bus for the OLED uses non-standard GPIO pins (14, 15) because the camera module occupies the default I2C pins (21, 22). `Wire.begin(15, 14)` re-maps the I2C peripheral accordingly. The I2S peripheral is assigned to port 1 (Audio audio(1)) since I2S port 0 is internally used by the OV2640 camera XCLK generator.

Fig. 2 shows the complete circuit diagram including the TP4056 LiPo battery charger for portable operation, the ESP32-CAM with all peripheral connections, the MAX98357A I2S amplifier driving an 8 Ω speaker, and the SSD1306 OLED display.

[Fig. 2: Complete circuit diagram — ESP32-CAM, MAX98357A I2S amplifier, SSD1306 OLED display, TP4056 LiPo charger, and 8 Ω speaker.]

Fig. 2. Complete circuit diagram of the AI Vision for Blind system.

VI. SOFTWARE DESIGN

A. Firmware Architecture

The firmware is written in Arduino C++ targeting the Espressif ESP32 Arduino core v3.2.1. It is structured as a single-file sketch with five functional modules: (1) Wi-Fi management, (2) camera interface, (3) Groq API client, (4) TTS and audio pipeline, and (5) OLED display driver.

B. Wi-Fi Resilience

A custom `ensureWiFi()` function is called before every network operation. It first attempts a fast 10-second auto-reconnect; if that fails, it executes a full `WiFi.begin()` with a 15-second timeout. The IDF-level call `esp_wifi_set_ps(WIFI_PS_NONE)` is invoked in every loop iteration and after every reconnect to permanently disable modem sleep, preventing the intermittent TCP connection drops common in power-saving mode [3].

C. Groq API Integration

The image description request is formatted as a JSON payload conforming to the OpenAI-compatible chat completions API. The payload includes the system prompt, a text instruction, and the Base64-encoded JPEG wrapped in a data URI (`data:image/jpeg;base64,...`). The Groq endpoint responds in $\approx 1.5\text{--}3$ s using the LLaMA-4 Scout 17B model, with `max_tokens` limited to 200 to keep response time predictable.

D. TTS Pipeline

The AI response is split into chunks of ≤ 200 raw characters at word boundaries to respect the Google Translate TTS URL length limit. Each chunk is URL-encoded and passed to `audio.connecttohost()`. The `audio_eof_stream()` callback sets a flag when a chunk finishes, triggering the next chunk. A 20-second wall-clock timeout ensures the loop always terminates, preventing infinite blocking if a chunk fails to load.

VII. WORKING PRINCIPLE

The complete operational sequence is as follows:

Step 1 — Button Detection: The main loop polls GPIO 3. A valid press is confirmed after a 50 ms debounce delay.

Step 2 — Wi-Fi Verification: `ensureWiFi()` is called; the pipeline aborts with a display message if connectivity cannot be established within 25 seconds.

Step 3 — Image Capture: GPIO 4 (flash LED) is asserted HIGH for 400 ms to illuminate the subject. Three warm-up frames are discarded before the final frame is captured into PSRAM. The LED is de-asserted, and the JPEG buffer is Base64-encoded.

Step 4 — API Request: The JSON payload is POSTed to the Groq API over HTTPS with a 30-second timeout. `http.setReuse(false)` ensures a fresh TCP connection per request, preventing stale socket issues on repeated captures.

Step 5 — Response Parsing: A lightweight string parser locates the "content" field within the "message" object without invoking a full JSON library on the large payload, conserving heap. Escape sequences (`\n`, `\r`, `\t`, `\"`) are normalized to plain text.

Step 6 — Simultaneous Audio-Visual Output: The response text is split into OLED display lines and TTS URL chunks. The audio stream starts immediately while the OLED shows the first page; subsequent pages advance every 1.5 s to approximate reading pace synchronized with speech.

Step 7 — Cleanup: After the 20 s timeout or a cancel event, `audio.stopSong()` is called, all large String objects (b64, payload, raw JSON) are explicitly cleared to reclaim heap, and the display returns to the main ready screen.

VIII. MATHEMATICAL ANALYSIS

A. Base64 Encoding Overhead

Base64 encoding represents every 3 bytes of binary data as 4 ASCII characters. For a JPEG image of size n bytes, the encoded output size S_{b64} is:

$$S_{b64} = \lceil n / 3 \rceil \times 4 \text{ bytes}$$

For a typical VGA JPEG of $n = 35,000$ bytes: $S_{b64} = \lceil 35000/3 \rceil \times 4 = 11667 \times 4 = 46,668$ bytes, representing a 33.3% size increase. This is the dominant factor in the HTTP payload size (~ 49 KB including JSON wrapper).

B. TTS Chunk Count

Given a response string of length L characters and a maximum chunk size of $C = 200$ characters, the number of TTS requests N_{tts} is:

$$N_{\text{tts}} = \lceil L / C \rceil$$

A typical two-sentence response contains 120–200 characters, requiring $N_{\text{tts}} = 1$ or 2 chunks. Each HTTP GET to Google TTS takes approximately 0.8–1.5 s over a standard broadband connection, giving a total TTS latency of 0.8–3.0 s.

C. End-to-End Latency

The total system latency T_{total} from button press to first audio output is the sum of individual stage latencies:

$$T_{\text{total}} = T_{\text{capture}}^{\text{ure}} + T_{\text{encode}}^{\text{de}} + T_{\text{api}}^{\text{I}} + T_{\text{parse}}^{\text{r}} + T_{\text{tts}}$$

Where: $T_{\text{capture}} \approx 0.5$ s (flash + warm-up + PSRAM read); $T_{\text{encode}} \approx 0.05$ s (Base64, 240 MHz); $T_{\text{api}} \approx 1.5$ –3.0 s (Groq inference + network RTT); $T_{\text{parse}} \approx 0.01$ s; $T_{\text{tts}} \approx 0.8$ –1.5 s (first chunk). Thus $T_{\text{total}} \approx 2.9$ –5.1 s to first audio, with full speech output completing within 8–12 s for a 160-character response.

D. Heap Memory Budget

The ESP32-S has 320 KB internal SRAM and 4 MB PSRAM. The camera frame buffer (VGA JPEG ~35 KB) is allocated in PSRAM. The Base64 string (~47 KB) is also PSRAM-allocated. After the API response (~0.5–1 KB text) is extracted, the large strings are explicitly freed. Maintaining >50 KB free internal heap is critical for SSL/TLS session buffers; the firmware clears b64 and payload immediately after the POST to preserve headroom.

IX. RESULTS AND DISCUSSION

The system was evaluated across three categories of scenes: (1) indoor objects (household items, books, electronics), (2) outdoor scenes (street views captured through a window), and (3) text-bearing surfaces (product labels, signage). Testing was conducted over 50 capture cycles across multiple Wi-Fi sessions.

The Groq/LLaMA-4 model produced accurate and contextually relevant descriptions in 88% of trials. Failure cases (12%) were primarily attributable to insufficient ambient lighting despite flash assistance, or highly cluttered scenes where the model described only part of the frame. Average end-to-end latency was 4.3 ± 0.9 seconds, consistent with the theoretical estimate from Section VIII.

Wi-Fi connectivity proved stable across 50 cycles with zero mid-session dropouts, validating the `ensureWiFi()` reconnect strategy. The 20-second TTS timeout prevented any instance of infinite blocking during intermittent Google TTS delays. The cancel mechanism responded within 800 ms in all tests.

Power consumption was measured at approximately 320 mA @ 5 V (1.6 W) during active capture/transmission and 95 mA during idle polling. A 10,000 mAh USB power bank provides an estimated 30+ hours of idle standby or approximately 6–8 hours of continuous use (one capture per minute).

X. CONCLUSION

This paper presented AI Vision for Blind, a low-cost, embedded assistive system enabling real-time auditory scene description for visually impaired users. By combining the ESP32-CAM's on-chip image capture capability with the Groq cloud API's state-of-the-art LLaMA-4 vision-language model and Google TTS, the system delivers contextually meaningful speech output without requiring any specialized computing hardware beyond a standard Wi-Fi network.

The system achieved an average end-to-end latency of 4.3 seconds, 88% description accuracy, and stable multi-session operation at a total hardware cost under ₹1,500. The single-button, no-screen-required interface makes it immediately deployable for users with no technical background.

Future work will explore edge-inference models (e.g., quantized LLaVA) for offline operation, GPS integration for outdoor navigation assistance, and a wearable form factor with a 3D-printed head-mounted enclosure. Multilingual TTS support and a companion smartphone app for configuration management are also planned.

REFERENCES

- [1] World Health Organization, "Visual Impairment and Blindness," WHO Fact Sheet No. 282, Oct. 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment>
- [2] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. Int. Conf. Machine Learning (ICML), 2021, pp. 8748–8763.
- [3] S. Venkatesan, R. Misra, and H. Gupta, "Smart Assistive System for Visually Impaired Persons Using IoT and Computer Vision," IEEE Internet of Things J., vol. 8, no. 14, pp. 11324–11334, Jul. 2021, doi: 10.1109/JIOT.2021.3054961.
- [4] A. Kumar, P. Singh, and D. Sharma, "Voice-Guided Navigation and Object Detection for Visually Impaired Users Using Deep Learning on Embedded Platforms," IEEE Access, vol. 10, pp. 47213–47228, 2022, doi: 10.1109/ACCESS.2022.3172108.
- [5] M. Zhang and L. Chen, "Real-Time Object Detection for Visually Impaired Persons Using Lightweight Mobile Neural Networks on IoT Devices," in Proc. IEEE Int. Conf. Consumer Electronics (ICCE), Las Vegas, NV, USA, Jan. 2023, pp. 1–5, doi: 10.1109/ICCE56470.2023.10043530.
- [6] R. K. Megalingam, S. M. Chintalacheruvu, and V. Rajesh, "Assistive Technology for Visually Impaired Using Android Smart Phone," in Proc. IEEE Int. Conf. Advanced Communications Technology (ICACT), 2020, pp. 338–342.